Evaluating classical edge detection algorithms under image degradation

Manana Sedrakyan

Individual Research Paper

Abstract..... 3
Introduction..... 3
Background..... 4
Methodology..... 6
Results..... 8
Discussion..... 12
Limitations..... 13
Conclusion..... 13
References..... 13

Abstract

This study investigates how image degradation and, more precisely, salt-and-pepper noise affect the quality and runtime of four main edge-detecting algorithms: Sobel, Prewitt, Laplacian and Canny. To test and find the answer to this question, the research uses Berkeley Segmentation Dataset and Benchmarks 500 (BSDS500). To find the change of quality and runtime, the application first calculates the precision and recall of the matching edges given from the dataset and detected edges from the images and uses them to find the quality representing F1. Also, the program counts the average execution time. The findings reveal the differences of how each algorithm responds to the image degradation by comparing the quality and runtime. For instance, Canny achieved the highest F1-score under both tested salt-and-pepper noise conditions, while Sobel demonstrated the smallest relative decline in F1-score as noise increased. The results demonstrate that algorithm performance depends on the type and severity of image degradation. Overall, this study provides an experimental comparison of classical edge-detection algorithms under different image degradation conditions and shows the importance of detection accuracy and computational performance when evaluating their effectiveness.

Introduction

Edges are important features of digital images and screens. They usually represent the image intensity, change of the materials or colors. Edge-detection is a technique of finding the sharp edges of the image, allowing the computer to allocate the borders of the objects on the screen. Therefore it is a fundamental technique in computer vision and digital image processing (Canny, 1986; Marr & Hildreth, 1980) It is widely used in the real world and in different fields. For example, edge-detection can assist doctors to detect in the medical images the diagnosis of diseases, like cancer. In addition, this technique can be used in game development to achieve a comic-book-like effect for the game. These applications demonstrate the importance of accurate edge-detection in both scientific and creative fields.

Despite the fact that edge-detection is a crucial part of the image processing, still the real-world images are not perfect and it becomes challenging to identify the borders. Usually, the input images are blurred, noisy or with changed contrast. Thus, the well-known edge-detecting algorithms may behave differently while examining the edges of the images and the borders may become less distinct. Consequently, the algorithms may respond differently for the same images. This study therefore examines the influence of imperfect images on the four famous edge-detecting algorithms. The purpose of investigation is to find out how the degraded images under effects of Gaussian blur, salt-and-pepper noise or changes in contrast affect the quality and execution time of Sobel, Prewitt, Laplacian and Canny algorithms. The goals of the research are implementing four edge-detection algorithms on classical and clean input images, evaluate their performance under previously mentioned

degraded images, compare the runtime and detection quality via quantitative metrics and finally identify strengths and weaknesses for each algorithm under different conditions. Before conducting the experiments, my main hypothesis is that Canny would be more robust to image degradation, particularly noise, and produce more accurate and continuous edges than the other three algorithms.

Background

To start out with, we should define the main concepts used in this study. Pixel is the smallest and fundamental building block of a digital screen or image, which stores a value or color. The intensity of a pixel can be represented as a two-dimensional function (call it I(x,y)), where x and y demonstrate the location of the pixel. The grayscale image represents the intensity of each pixel by using shades of black to white (0 is black and 255 is white). An edge occurs when there is a significant change between neighboring pixels. This edge often identifies the boundary of different objects. One of the main mathematical concepts behind the edge detection is image gradient, which measures how quickly the image intensity changes in the horizontal and vertical direction and it consists of two main partial derivatives.

$$G_x = \frac{\partial I}{\partial x}, G_y = \frac{\partial I}{\partial y}$$

To calculate the intensity of the change, we should calculate the gradient magnitude

$$G = \sqrt{G_x^2 + G_y^2}$$

If the magnitude is large, it represents a stronger intensity and hence a possible edge(Canny, 1986). For each of our edge detecting algorithms, we will create a thresholded value, which is a selected value to compare with the magnitude. If the magnitude exceeds the threshold, then the pixel will be classified as an edge and vice versa.

Each of the algorithms uses a predefined kernel, which is a matrix of numerical weights that is used to perform image-processing operations. An operation used to combine the input image and kernel is called a convolution, which is when the kernel moves across the image and calculates weighted sums of neighboring pixel values. The other, multiplying overlapping values and summing them to get the third function.

➢ Sobel and Prewitt operators

Both operators are gradient-based edge-detecting algorithms that estimate intensity changes in horizontal and vertical directions. They use 3x3 convolution kernels , one for each direction. In our study, the kernels for the Sobel algorithms are as follows.
For Sobel in X direction:

```
    {{-1,0,1},
    {-2,0,2},
    {-1,0,1}
```

For Sobel in Y direction:

```
    {{-1,-2,-1},
```

```
    { 0,0,0},
    { 1,2,1}}
```

The results after these two kernel convolutions with the original image show the horizontal and vertical intensity changes. Then the magnitude is calculated and finally the threshold is applied to produce a binary image map. Prewitt method is similar to Sobel, however it uses equal weights in its kernels. In contrast, Sobel uses greater weight to the central row and column (Sobel & Feldman, 1968; Prewitt, 1970).
For Prewitt X direction:

```
{{-1,0,1},
{-1,0,1},
{-1,0,1}}
```

For Prewitt Y direction:

```
{{-1,-1,-1},
{ 0,0,0},
{ 1,1,1}}
```

➢ Laplacian operator

The Laplacian operator is based on a second order derivative of the image function. It measures changes in the image gradient and can therefore identify regions where intensity changes rapidly. Common kernel used in the study is following:

```
{{0,1,0},
{1,-4,1},
{0,1,0}}
```

The Laplacian algorithm produces rapid intensity change in the image. Then the edges can be identified when the function changes signs. However, the Laplacian algorithm is sensitive to the noise, because the second-derivative often produces small intensity variations. That is why for some implementations at first a Gaussian smoothing was applied before calculating the Laplacian algorithm(Marr & Hildreth, 1980).

➢ Canny operator

This operator is a mixed approach that combines Gaussian smoothing and Sobel algorithm. The image is run through different operations step by step. The program applies Gaussian smoothing to the image and then calculates the gradient of the Image. After, the output is run through non maximum suppression, which is a program to remove pixels that are not local maxima in the gradient direction. The program keeps only the strongest intensities, so the output produces the accurate and thin edges. To classify the strongest and weakest edges, the program runs double thresholding with lower and upper thresholds, to find the weaker and stronger edges. And the final stage is tracking the edges by hysteresis. Weak edges are examined to determine whether they are connected to strong edges. Weak pixels connected to strong edges are preserved, while isolated weak pixels are removed. This approach is used for preserving connected edges and removing false detections. This complex approach results in more selective edge detection(Canny, 1986).

Edge detection was always an important part of image processing and visual computing. There were different early theoretical researches that established the connection of edge detection and mathematical approaches. For example, Marr and Hildreth proposed a theory based on the second derivative of a Gaussian, demonstrating the importance of considering image scale and smoothing when detecting edges(Marr & Hildreth, 1980). Another contribution was done by John Canny, who introduced a more systematic approach to edge detection based on detection accuracy, localization, and minimizing multiple responses to a single edge. His work created a theoretical foundation for the Canny detector (Canny, 1986). To test the work of the algorithms, it is important to have a dataset of images. Martin et al. introduced the Berkeley Segmentation Dataset and Benchmark 500, which contains natural images with human-provided boundary annotations. This research uses images from BSDS500 to examine the performance of the four implemented algorithms (Martin et al., 2001).

Methodology

The experiments use the Berkeley Segmentation Dataset and Benchmark 500 (BSDS500). It is a dataset containing 500 natural images with their human-generated segmentation and boundary annotations. The dataset contains testing, validation and training subsets. This dataset was selected because each image had its reference boundary annotations that allow automatically detected edges to be compared against human interpretations. This makes it suitable for evaluating the detection quality of different edge-detection algorithms (Martin et al., 2001). Convenient 10 images were selected from the dataset. The corresponding ground-truth annotations were obtained from the dataset's annotation files and converted into PNG images for use in the Java application. For each image, only the first available human annotation was used as the reference boundary map. For each algorithm there were conducted five main experimental conditions: without any change (Original), random black and white pixels affected 5% or 10% of the image (Salt-and-Pepper noise 5% and 10%, correspondingly 2.5% and 5% of salt and pepper), blur used Gaussian smoothing(Gaussian blur) and image with reduced contrast using a factor of 0.5 (Reduced contrast).
Gaussian blur and smoothing use the following kernel.

```
{{1.0/256, 4.0/256, 6.0/256, 4.0/256, 1.0/256},
{4.0/256, 16.0/256, 24.0/256, 16.0/256, 4.0/256},
{6.0/256, 24.0/256, 36.0/256, 24.0/256, 6.0/256},
{4.0/256, 16.0/256, 24.0/256, 16.0/256, 4.0/256},
{1.0/256, 4.0/256, 6.0/256, 4.0/256, 1.0/256}}
```

Contrast reduction was calculated with the following formula:

```
double adjusted =  contrast * (pixel - 128) + 128;
```

The resulting intensity values were restricted to the valid grayscale range of 0 to 255.

The experiment consisted of loading the original images, preparing them for processing, generating the five experimental conditions, applying the four edge-detection algorithms and finally evaluating their outputs against the corresponding reference boundaries. Each degraded image was generated before the detector was executed, so that all four algorithms were evaluated on the same input condition. In total, the experiment consisted of 10 images, five conditions, and four algorithms, resulting in 200 image-condition-algorithm combinations.

The whole application was implemented in Java. For each of the four algorithms there exists a separate class. The same clean or degraded input images were used to compare their performance. The Sobel detector calculates horizontal and vertical intensity changes using two 3x3 convolution kernels. The resulting gradient magnitude is compared against a threshold to produce a binary edge map. The threshold used in the experiment was 50. The same threshold and approach is used also for the Prewitt algorithm. The Laplacian detector uses a second-order derivative kernel to identify rapid intensity changes. The resulting response is processed using the detector's thresholding procedure, with a threshold of 50. The Canny detector uses a multi-step algorithm consisting of Gaussian smoothing, gradient calculation, non-maximum suppression, double thresholding, and edge tracking by hysteresis. The lower threshold was set to 50 and the upper threshold to 100. These parameters were kept consistent across all experimental conditions.

The quality of each edge-detection result was evaluated by comparing the detected edge pixels with the corresponding ground-truth boundary annotation. There are three quantitative metrics: Precision, Recall and F1-score. Precision measures the proportion of detected edge pixels that correctly correspond to reference boundary pixels. It is calculated as:

```
TP/(TP + FP);
```

where TP represents true positive matches and FP represents false positive detections. Recall measures the proportion of reference boundary pixels that were successfully detected by the algorithm. It is calculated as:

```
TP / (TP + FN);
```

where TP represents true positive matches and FN represents boundary pixels that were not detected.

F1-score combines precision and recall into a single metric using their harmonic mean:

```
2 * (precision * recall) / (precision + recall);
```

A higher F1-score indicates a better balance between precision and recall. This metric was used to compare the overall detection quality of the four algorithms across the different conditions.

To account for small differences in the positions of detected and reference boundaries, the application used a boundary-matching procedure. A detected edge pixel was considered a possible match if its Euclidean distance from a reference boundary pixel was no greater than one pixel. The matching procedure used one-to-one correspondence, meaning that a detected pixel and a reference pixel could each participate in at most one match. The implementation selected matches to maximize the number of valid one-to-one correspondences. This tolerance prevents a single reference pixel from being counted as multiple successful detections.

The next evaluation metric is the runtime. To reduce the influence of initial execution overhead, five warm-up runs were performed before the measured runs. After warm-up, each detector was executed 20 times on the same input image. The measurements were used to calculate the average runtime, median runtime and standard deviation. Only the execution of the edge-detection algorithm was included in the timing measurements. Image loading, image degradation, and file-saving operations were excluded. This allowed us to focus on the runtime of each algorithm. The runtime measurements were recorded in the experimental results and used to compare the computational efficiency of Sobel, Prewitt, Laplacian and Canny.

Results

The evaluation of the four algorithms under experimental conditions focuses on two main aspects: edge-detection quality, measured using precision, recall, and F1-score and computational performance, measured using execution time.

Figure 1 presents the average F1-scores obtained by the four algorithms under the different experimental conditions.

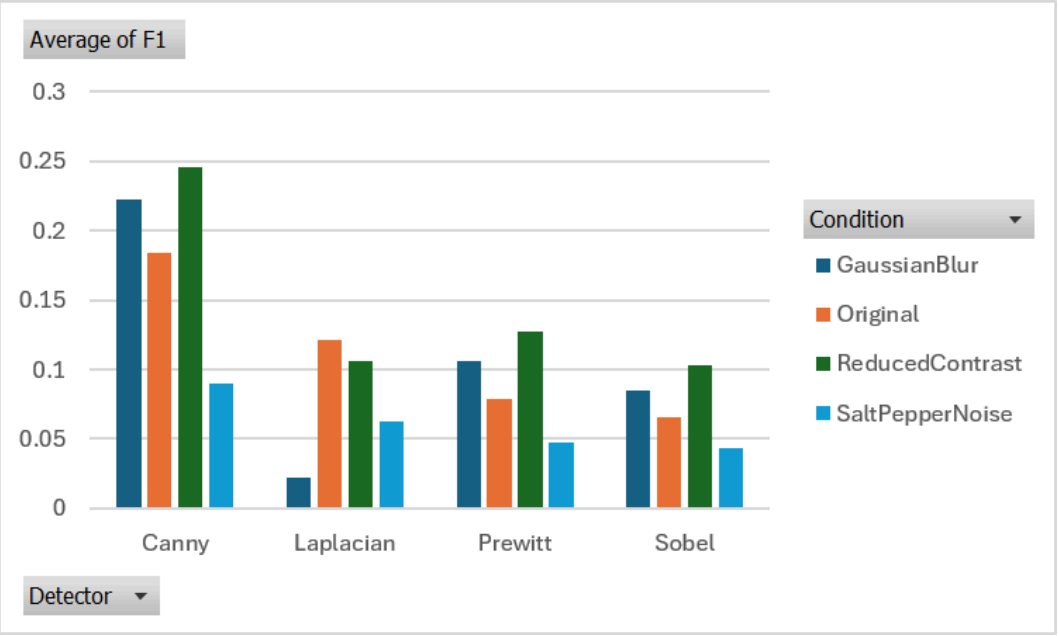


Figure 1

The results demonstrate differences in edge-detection performance between the four algorithms. Based on the experimental results, Canny achieved the highest overall F1-score under the tested conditions, while Laplacian obtained the lowest score. The results also show that the performance of the algorithms varied depending on the type of image degradation.

Figure 2 compares the average execution time of the four edge-detection algorithms across the experimental conditions.

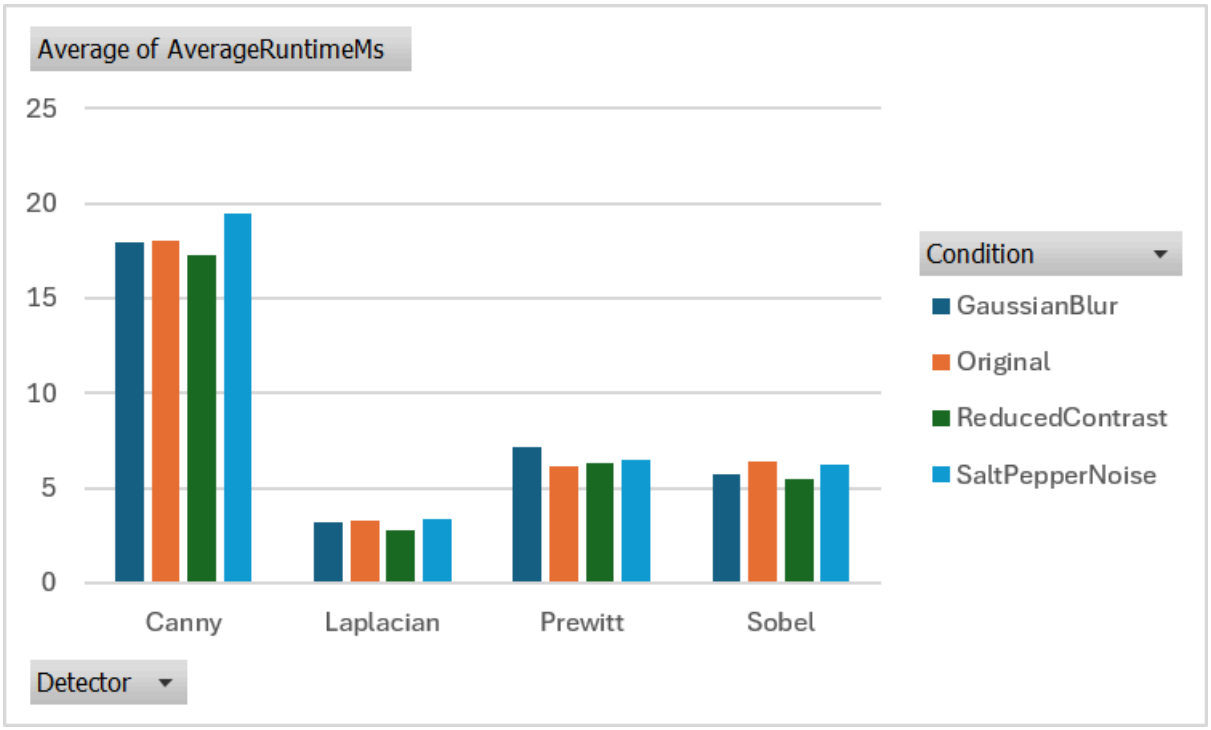


Figure 2

The measurements show that Laplacian had the lowest average execution time, whereas Canny required the most processing time. The runtime comparison reveals differences in computational cost between the algorithms. In particular, Canny involves several processing stages, including Gaussian smoothing, gradient calculation, non-maximum suppression, double thresholding, and edge tracking by hysteresis. The simpler gradient-based methods involve fewer processing stages. These results provide a comparison of computational

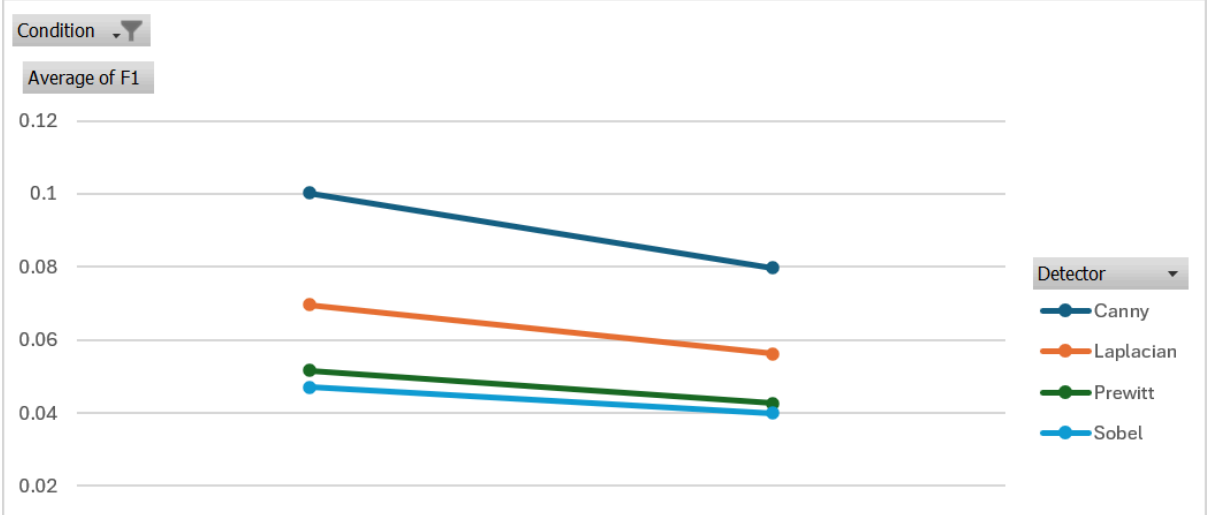


Figure 3

performance in addition to detection quality. However, the measured execution times represent only the execution of the detection algorithms and does not include image loading, degradation, and file-saving operations. Figure 3 illustrates the relationship between noise severity and the average F1-score of each algorithm. The algorithms were tested using 5% and 10%. Canny achieved the highest F1-score at both noise levels, with a score of 0.1002 at 5% noise and 0.0797 at 10% noise. This indicates that it produced the strongest detection performance among the four tested algorithms under these two conditions. However, the relative decrease in F1-score was different for each algorithm. Sobel demonstrated the smallest relative decrease, at approximately 15.0%, while Canny experienced a decrease of approximately 20.4%. Therefore, the algorithm with the highest F1-score was not the algorithm with the smallest relative performance decline. Canny maintained the highest detection quality, whereas Sobel showed the smallest proportional decrease as noise increased.

Figure 4 compares precision and recall for the four algorithms on the original, non-degraded images.

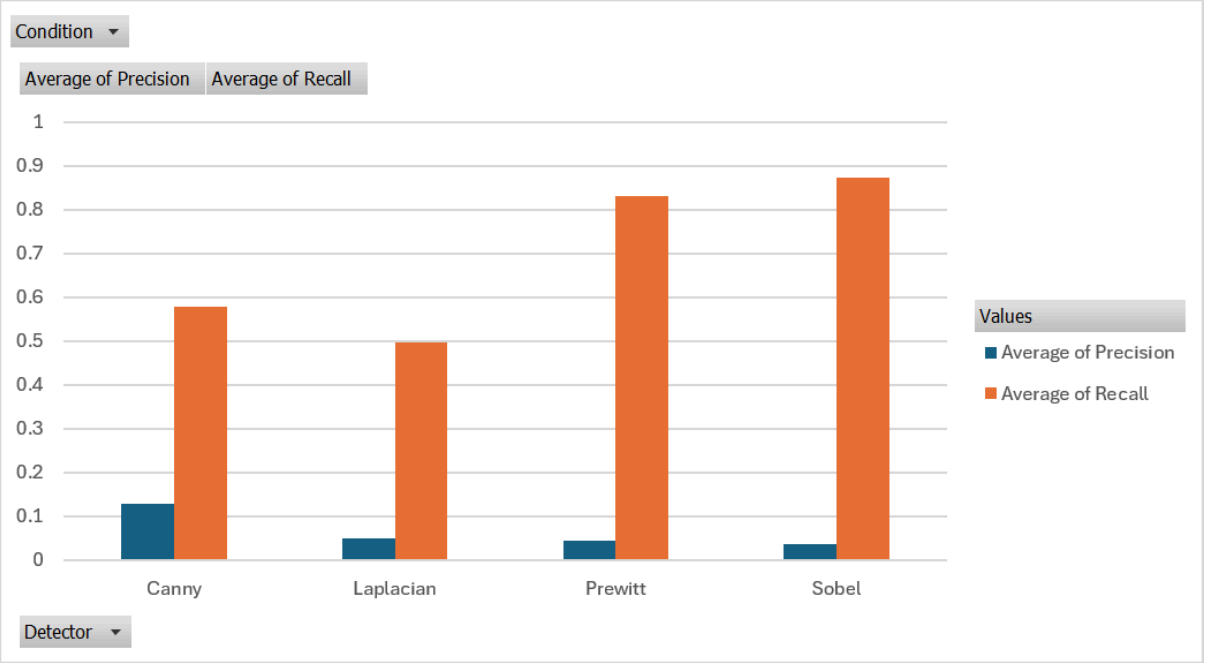
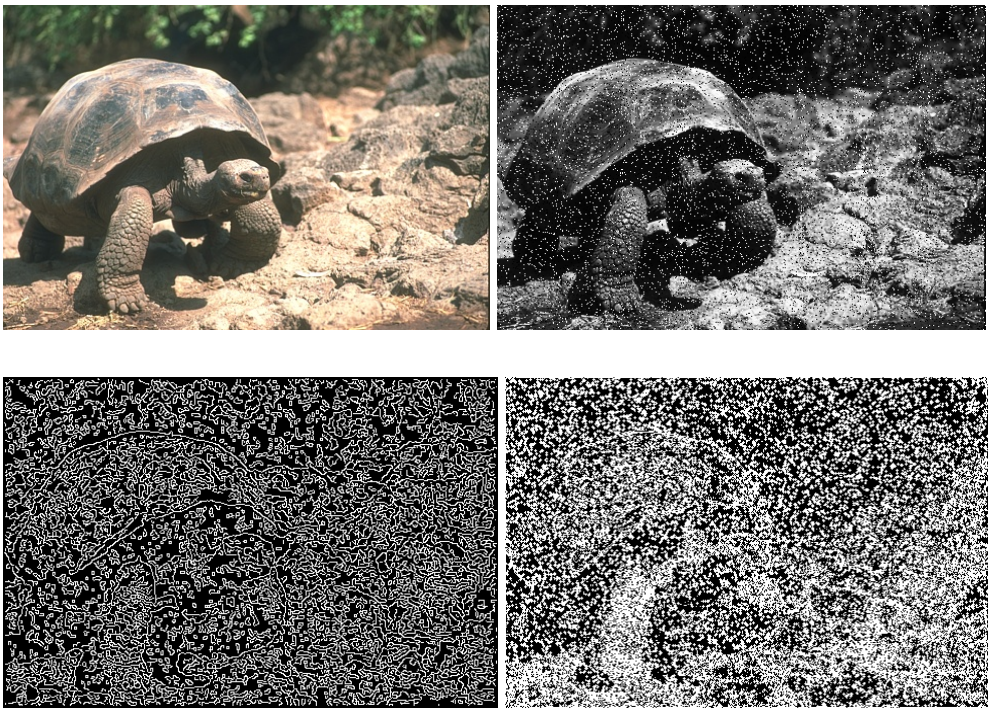


Figure 4

The comparison provides additional information about the types of detection errors made by each algorithm. Precision shows the proportion of detected edge pixels that correspond to reference boundaries, while Recall measures the proportion of reference boundary pixels successfully detected. The results show that Sobel achieved the highest Precision, while Canny achieved the highest Recall. These differences indicate that the algorithms vary in the false-positive edge detections and identifying reference boundaries.

It is worth mentioning that there were visual differences as well. From the figure 5, we can see the same image (103006.jpg) under noisy conditions (10%) through all four algorithms.



Canny Laplacian

Prewitt Sobel

Figure 5

Discussion

Different experiments in this study showed that for each degraded image the algorithm's result varied and the algorithms didn’t respond identically to the degraded image condition. One of the important findings was that Canny achieved the highest F1-score under both tested salt-and-pepper noise levels. Its average F1-score decreased from 0.1002 at 5% noise to 0.0797 at 10% noise. This suggests that Canny held the strongest detection quality

among the four tested algorithms under these conditions. However, Sobel had the smallest relative decrease in performance (15.0%), compared with Canny's 20.4%. This means that although Canny achieved better detection quality, Sobel's results changed less when the noise level increased. The runtime comparison showed that Laplacian was the fastest, while Canny required the most processing time. This is expected to some extent because Canny involves more processing stages than simpler methods such as Laplacian, Sobel orPrewitt. It also shows why detection quality should not be the only factor considered when comparing algorithms. Overall, the results partly supported the initial hypothesis that Canny would perform better under degraded conditions. It achieved the highest F1-score in the noise experiments, but it did not have the smallest relative performance decline. The findings suggest that choosing an edge-detection algorithm depends on the specific image conditions and whether detection quality or computational efficiency is more important.

Limitations

This study has several limitations that should be considered when viewing the results. First, only ten images from BSDS500 were tested, which may not represent the full variety of natural images. Second, only the first human boundary annotation was used, although the dataset provides multiple annotations. Another limitation is that a custom boundary-matching method was used instead of the official BSDS500 evaluation protocol. Therefore, the results should not be treated as official benchmark scores. Finally, only one blur and one contrast-reduction level were tested. The results may also depend on the selected thresholds and implementation choices.

Conclusion

This study compared four classical edge-detection algorithms under different image degradation conditions. In the noise experiments, Canny achieved the highest F1-scores, while Sobel had the smallest relative performance decrease. This shows that the algorithm with the best detection quality is not necessarily the one with the most stable results. Overall, the study demonstrates the importance of considering both detection quality and computational performance when choosing an edge-detection algorithm. For future study, it is important to include more images, different human annotations, various blur and contrast levels.

References

Arbeláez, P., Maire, M., Fowlkes, C., & Malik, J. (2011). Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 33*(5), 898–916.

Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-8*(6), 679–698. https://doi.org/10.1109/TPAMI.1986.4767851

Marr, D., & Hildreth, E. (1980). Theory of edge detection. *Proceedings of the Royal Society of London. Series B, Biological Sciences, 207*(1167), 187–217. https://doi.org/10.1098/rspb.1980.0020

Martin, D., Fowlkes, C., Tal, D., & Malik, J. (2001). A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. *Proceedings of the Eighth IEEE International Conference on Computer Vision*, 2, 416–423. https://doi.org/10.1109/ICCV.2001.937655

Prewitt, J. M. S. (1970). Object enhancement and extraction. In B. S. Lipkin & A. Rosenfeld (Eds.), *Picture processing and psychopictorics* (pp. 75–149). Academic Press.

Sobel, I., & Feldman, G. (1968). *A 3×3 isotropic gradient operator for image processing*. Stanford Artificial Intelligence Project. (Unpublished presentation.)